

p0902r0

Move-only iterators

Published Proposal, 5 February 2018

This version:

<http://wg21.link/P0902R0>

Author:

[Andrew Hunter](#) (Google)

Audience:

LEWG

Project:

ISO JTC1/SC22/WG21: Programming Language C++

Abstract

Nothing really requires iterators to be copyable. Can we remove that restriction?

Table of Contents

1	Motivation
1.1	Sidebar: why might an iterator be noncopyable?
2	Proposal
2.1	What about <code>OutputIterator</code>
2.2	Alternative: can we retrofit?
	References
	Informative References

§ 1. Motivation

Modern C++ has left `InputIterator` in a curious position. We require it (through the definition of `Iterator`) to be copyable, but precious little can be done with any copied iterator; only one of a set of copies can be dereferenced. Why then does it require copyability?

```
template <class InputIt, class OutputIt, class UnaryOperation>
OutputIt transform(InputIt first1, InputIt last1, OutputIt d_first,
                  UnaryOperation unary_op) {
    return __transform_impl(first1, last1, d_first, unary_op);
}
```

The above code wants to implement `transform` by the use of a helper function, but calling that helper copies the iterator. There's an obvious way to remove that copy:

```
template <class InputIt, class OutputIt, class UnaryOperation>
OutputIt transform(InputIt first1, InputIt last1, OutputIt d_first,
                  UnaryOperation unary_op) {
    return __transform_impl(std::move(first1), std::move(last1), d_first, unary_op);
}
```

but that feature didn't exist when `Iterator` and `InputIterator` were created. I wasn't there, but I'll bet this is why the requirement was specified.

I'd like to undo that decision (to the extent that is possible.)

§ 1.1. Sidebar: why might an iterator be noncopyable?

Why does this matter? Consider a reader-writer data structure (my examples all derive from extensions of `[Cell]`) where we expose protected data that can only be safely held while holding a read lock. `cell` and friends ensure this by only allow users to see data contained in RAII types (greatly simplified here):

```

Token ReadLock();
void ReadUnlock(Token t);

template <typename T>
class snapshot {
public:
    const T& operator*() const { return *p_; }
    ~snapshot() { ReadUnlock(t_); }

    snapshot(snapshot<T> &&rhs) { /* ... */ }
    snapshot(const snapshot<T> &rhs) = delete;
private:
    snapshot(T* p, Token t) : p_(p), t_(t) {}
    T* p_;
    Token t_;
};

```

The constructor (only used by `cell`'s implementation) ensures that a reader lock is held protecting the exposed data; and the `snapshot` allows safe access to the locked data: it is not possible (without trying quite hard) to accidentally unlock too early, nor forget to unlock (so long as you destroy your snapshots.)

Snapshots are noncopyable first and foremost for safety (there's no good reason to copy them) but a more important limit is correctness. In some major RCU-based implementations (including our high-performance implementation), just because we *hold* the read lock protecting some data does not mean we can *take* that same read lock again. (This has to do with our safe disposal mechanism: we have to guarantee that no *new* snapshots will be created referencing some data.)

So consider a data structure that contains multiple cell-like pieces of data, like a concurrent `unordered_map`: the iterators must contain `snapshot`s of the items, but we *cannot* copy them. There is a potential workaround:

```

template <typename T>
class Iterator {
    Iterator(snapshot<T> snap) : shared_(std::move(snap)) {}
private:
    std::shared_ptr<snapshot<T>> shared_;
};

```

But this is absurd--there is no good reason to hold a shared pointer to what is morally just a special a `unique_ptr`, and this requires allocation on every use of iterators.

At Google we use these `cell`-like map types and often need to iterate them; we'd very much like for them to have a valid iterator type for use in range-for constructs; it would also simplify their lookup APIs if we could match `std::unordered_map`. But without unreasonable workarounds, it is not possible for use to create even an `InputIterator` for these types as they are fundamentally uncopyable.

§ 2. Proposal

I propose a fairly simple fix. Informally (this isn't intended to be final wording) we would:

1. strike `CopyConstructible` from the requirements in `[iterator.iterators]`
2. Add a concept `MoveIterator`, with the current text of `[input.iterators]`
3. Define `InputIterator` as a `CopyConstructible MoveIterator`.

(With some obvious changes elsewhere adding `MoveIterator` to the various lists of kinds of iterator.)

We edit `InputIterator` to ensure all current code that creates and uses iterators maintains precisely the same requirements (but see [§2.2 Alternative: can we retrofit?](#) for another possibility.) The new concept is free for anyone to create algorithms with, and can be applied to data structures for both their access APIs and use of range-for iteration.

A much larger extension would be retrofitting `MoveIterator` across the standard library: the vast majority of iterator algorithms can obviously accept a `MoveIterator` with no serious implementation difficulty, but it's not clear which *implementations*, if any, have innocent but unnecessary copies within (say) `std::copy`. In this paper I propose we simply punt on this and do not change the required iterator tags on any algorithm. The ability to have honest-to-goodness `Iterator` concepts on our data structures and use range-for syntax gets us quite far already. If this proves successful and there is demand for `std::copy` or `std::transform` or anything else we can add it then.

Another question is iterator tags. The natural thing would be to redefine:

```
struct move_iterator_tag { };
struct input_iterator_tag : public move_iterator_tag { };
```

The biggest problem here is that algorithms that are currently guaranteed to work (and have explicit overloads for) any C++17 iterator category will no longer work for any iterator, until they are updated to have `move_iterator_tag` overloads. However, it is important to emphasize that this alone will not break any user code. User code will continue to work with any concrete iterator type that it worked with before, and will not break unless some part of the program is changed to produce a `MoveIterator` instead of an `InputIterator`. (This becomes more of an issue if we retroactively downclassify standard library iterators to `MoveIterator`s, but as we make clear above, we don't propose doing so unless and until pressing need arises.)

§ 2.1. What about `OutputIterator`

Much of the same argument above applies to `OutputIterator`: it's required to be copyable, but there's not much that can be done with it. It's very possible we should apply the same transformation.

As our use cases all revolve around read access, I've left out any attempt to do this just to keep things shorter. I have no strong opinion for or against.

§ 2.2. Alternative: can we retrofit?

There's a more ambitious choice we could make here: strike `CopyConstructible` from `Iterator`, and add it to `ForwardIterator` (which clearly needs it!) instead of `InputIterator`, without adding any new class of iterators whatsoever. This is tempting! My entire argument is that any reasonable use of `InputIterator` doesn't need to copy, and in principle this is backwards-compatible: all extant `InputIterators` are still perfectly good, as is all extant code that uses them.

The problem is the same retrofitting workload I want to avoid that explains why I don't think we should change the standard library's iterator requirements yet: code that used the new less restrictive definition of `InputIterator` would want to use extant algorithms on those iterators, and there's no guarantee at all that any implementation would support that. What's worse, that applies to both standard library algorithms *and* any user-created algorithm on `InputIterators` that contains copies.

I think this is worth considering, but probably not practical.

§ References

§ Informative References

[Cell]

Andrew Hunter; Geoffrey Romer. [An RAI Interface for Deferred Reclamation](http://wg21.link/P0561R3). November 11, 2017. URL: <http://wg21.link/P0561R3>